

AI complaint analyser developed using python and typescript

A modern, full-stack web application for managing customer complaints and product batches in a Quality Management System (QMS). Built with FastAPI backend and React/TypeScript frontend, featuring AI-powered complaint processing and analysis.

Table of Contents

- [Overview](#)
- [Features](#)
- [Tech Stack](#)
- [Project Structure](#)
- [Prerequisites](#)
- [Installation & Setup](#)
- [Running the Application](#)
- [API Endpoints](#)
- [Database Schema](#)
- [AI Complaint Processing](#)
- [Frontend Components](#)
- [Configuration](#)

Overview

AIVOA QMS is a comprehensive complaint management system designed for pharmaceutical and quality management operations. The system enables users to:

- **Create and manage customer complaints** with detailed information
- **Track product batches** and their manufacturing details
- **Manage customer profiles** and product inventory
- **Leverage AI agents** to automatically extract, categorize, and prioritize complaints from multiple sources
- **Process documents** (PDF, DOCX) containing complaint information
- **AI-assisted data correction** to ensure complaint accuracy

The application features a split-view interface with a complaint form on the left and an AI Copilot on the right for intelligent assistance.

Features

Complaint Management

- Create new complaints with comprehensive details
- Track complaint status (Pending Triage, In Progress, Resolved, etc.)
- Categorize complaints by type and severity
- Risk assessment and priority assignment

- Support for multiple complaint sources

Product & Batch Tracking

- Manage product inventory with dosage forms, strength, and specifications
- Track batch information including manufacturing and expiry dates
- Link complaints to specific product batches
- Query complaints by batch number

Customer Management

- Store customer information (name, email, phone, address)
- Track customer complaint history
- Support for different customer types

AI-Powered Features

- **Complaint Processing:** Automatically extract structured complaint data from unstructured text
- **Document Analysis:** Extract text from PDF and DOCX files
- **Data Correction:** AI-assisted correction and validation of complaint data
- **LLM Integration:** Uses Groq's language models via LangChain for intelligent processing

User Interface

- Real-time form validation
- Interactive complaint form with dropdown selections
- AI Copilot chat interface
- Error boundary for robust error handling
- Redux state management for predictable data flow

🔧 Tech Stack

Backend

- **FastAPI** (v0.110.0+) - Modern Python web framework
- **Uvicorn** - ASGI web server
- **SQLAlchemy** (v2.0.0+) - ORM for database management
- **Pydantic** (v2.6.0+) - Data validation
- **LangChain & LangGraph** - AI/LLM orchestration
- **Groq API** - Large language models
- **PyPDF** - PDF document processing
- **MySQL** - Relational database

Frontend

- **React** (v18.3.1) - UI library
- **TypeScript** - Type-safe JavaScript
- **React Redux** - State management
- **Axios** - HTTP client
- **React Router** - Client-side routing

- **date-fns** - Date manipulation
- **React Scripts** - Build tooling

Project Structure

```

Python_react/
├── backend/                                # FastAPI backend application
│   ├── main.py                            # FastAPI app initialization & CORS setup
│   ├── models.py                         # SQLAlchemy ORM models
│   ├── schemas.py                        # Pydantic validation schemas
│   ├── crud.py                           # Database CRUD operations
│   ├── agents.py                         # AI agent logic & LLM integration
│   ├── seed.py                           # Database seed/initialization
│   ├── requirements.txt                   # Python dependencies
│   └── app/
│       ├── __init__.py
│       └── routers/                      # API route handlers
│           ├── customers.py               # Customer endpoints
│           ├── products.py               # Product endpoints
│           ├── batches.py                 # Batch endpoints
│           ├── complaints.py              # Complaint CRUD endpoints
│           └── ai.py                      # AI processing endpoints
├── frontend/                              # React/TypeScript application
│   ├── package.json                      # NPM dependencies & scripts
│   ├── tsconfig.json                     # TypeScript configuration
│   ├── public/                           # Static assets
│   │   └── index.html                    # HTML entry point
│   ├── src/
│   │   ├── index.tsx                     # React app entry
│   │   ├── index.css                     # Global styles
│   │   ├── components/
│   │   │   ├── App.tsx                   # Main app component (split view)
│   │   │   ├── ErrorBoundary.tsx         # Error handling wrapper
│   │   │   └── form/
│   │   │       └── ComplaintForm.tsx      # Complaint form component
│   │   ├── copilot/
│   │   │   └── AICopilot.tsx             # AI assistant panel
│   │   └── ui/                           # Reusable UI components
│   ├── services/
│   │   └── api.ts                         # API client & HTTP requests
│   ├── store/
│   │   ├── store.ts                      # Redux store configuration
│   │   ├── hooks.ts                      # Custom Redux hooks
│   │   └── slices/
│   │       ├── complaintSlice.ts          # Complaint state management
│   │       └── uiSlice.ts                 # UI state management
│   └── build/                            # Compiled frontend (production)
└── readme.md                             # This file

```

Prerequisites

Before you begin, ensure you have the following installed:

- **Python 3.8+** - Backend runtime
- **Node.js 16+** - Frontend runtime
- **npm** or **yarn** - JavaScript package manager
- **MySQL 5.7+** - Database server
- **Git** - Version control

Installation & Setup

1. Clone & Navigate to Project

```
cd Python_react
```

2. Set Up Backend

Create Virtual Environment

```
# Windows (PowerShell)
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# macOS/Linux
python3 -m venv .venv
source .venv/bin/activate
```

Install Dependencies

```
cd backend
pip install -r requirements.txt
```

Configure Environment Variables

Create a `.env` file in the `backend/` directory:

```
# Database Configuration
DATABASE_URL=mysql+pymysql://username:password@localhost:3306/qms

# LLM Configuration (Groq API)
GROQ_API_KEY=your_groq_api_key_here

# Optional: LangChain Configuration
```

```
LANGCHAIN_API_KEY=your_langchain_key_here  
LANGCHAIN_TRACING_V2=false
```

Initialize Database

```
python seed.py
```

This will:

- Create the database if it doesn't exist
- Create all tables based on models
- Populate sample data (if available)

3. Set Up Frontend

```
cd ../frontend  
npm install  
# or  
yarn install
```

Configure API Endpoint

The frontend is configured to connect to <http://localhost:8000> by default. If using a different backend URL, update the API client in [src/services/api.ts](#).

Running the Application

Start Backend Server

```
cd backend  
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

The API will be available at: <http://localhost:8000> API Documentation (Swagger UI):
<http://localhost:8000/docs>

Start Frontend Development Server

In a new terminal:

```
cd frontend  
npm start
```

The application will open at: <http://localhost:3000>

Build for Production

```
# Frontend build
cd frontend
npm run build

# Backend is production-ready with Uvicorn
```

🔌 API Endpoints

All endpoints are prefixed with [/api/v1/](#)

Complaints

- [POST /complaints/](#) - Create a new complaint
- [GET /complaints/](#) - List all complaints (paginated)
- [GET /complaints/{complaint_id}](#) - Get complaint details
- [PUT /complaints/{complaint_id}](#) - Update complaint
- [GET /complaints/by-batch/{batch_number}](#) - Get complaints for a batch

Customers

- [POST /customers/](#) - Create customer
- [GET /customers/](#) - List customers
- [GET /customers/{customer_id}](#) - Get customer details
- [PUT /customers/{customer_id}](#) - Update customer
- [DELETE /customers/{customer_id}](#) - Delete customer

Products

- [POST /products/](#) - Create product
- [GET /products/](#) - List products
- [GET /products/{product_id}](#) - Get product details
- [PUT /products/{product_id}](#) - Update product
- [DELETE /products/{product_id}](#) - Delete product

Batches

- [POST /batches/](#) - Create batch
- [GET /batches/](#) - List batches
- [GET /batches/{batch_number}](#) - Get batch details
- [PUT /batches/{batch_number}](#) - Update batch
- [DELETE /batches/{batch_number}](#) - Delete batch

AI Processing

- **POST** `/ai/process` - Process complaint text with AI

```
{
  "input_text": "Customer complaint text...",
  "input_type": "text"
}
```

- **POST** `/ai/correct` - Apply AI-assisted data correction
- **POST** `/ai/upload` - Upload and process document (PDF/DOCX)

Health Check

- **GET** `/health` - Service health status



Database Schema

Customers Table

- **id** (INT, PK) - Auto-increment customer ID
- **name** (VARCHAR) - Customer name
- **email** (VARCHAR) - Email address
- **phone** (VARCHAR) - Phone number
- **address** (TEXT) - Street address
- **type** (VARCHAR) - Customer type/category
- **created_at** (DATETIME) - Creation timestamp

Products Table

- **id** (INT, PK) - Auto-increment product ID
- **name** (VARCHAR) - Product name
- **description** (TEXT) - Product description
- **type** (VARCHAR) - Product type
- **dosage_form** (VARCHAR) - Dosage form (tablet, liquid, etc.)
- **strength** (VARCHAR) - Strength specification
- **unit_of_measure** (VARCHAR) - Unit of measure
- **created_at** (DATETIME) - Creation timestamp

Batches Table

- **batch_number** (VARCHAR, PK) - Unique batch identifier
- **product_id** (INT, FK) - Reference to product
- **product_name** (VARCHAR) - Product name
- **strength_grade** (VARCHAR) - Batch strength grade
- **manufacturing_date** (DATE) - Date manufactured
- **expiry_date** (DATE) - Expiration date
- **quantity_manufactured** (VARCHAR) - Quantity produced
- **created_at** (DATETIME) - Creation timestamp

Complaints Table

- `id` (INT, PK) - Auto-increment complaint ID
- `complaint_source` (VARCHAR) - Source of complaint
- `customer_id` (INT, FK) - Reference to customer
- `customer_name` (VARCHAR) - Customer name
- `product_id` (INT, FK) - Reference to product
- `product_name` (VARCHAR) - Product name
- `product_strength` (VARCHAR) - Product strength
- `batch_number` (VARCHAR) - Associated batch
- `manufacturing_date` (VARCHAR) - Batch manufacturing date
- `expiry_date` (VARCHAR) - Batch expiry date
- `affected_quantity` (VARCHAR) - Quantity affected
- `complaint_category` (VARCHAR) - Category/type of complaint
- `complaint_date` (VARCHAR) - When complaint was reported
- `description` (TEXT) - Detailed complaint description
- `severity` (VARCHAR) - Severity level
- `priority` (VARCHAR) - Priority level
- `risk_assessment` (TEXT) - Risk assessment details
- `status` (VARCHAR) - Current status (default: "Pending Triage")
- `created_at` (DATETIME) - Creation timestamp



AI Complaint Processing

The system uses LangChain and Groq's language models to provide intelligent complaint processing.

Agents (`agents.py`)

`process_complaint(input_text: str)`

- Accepts unstructured complaint text
- Extracts structured data (customer, product, batch, etc.)
- Categorizes complaint
- Assigns severity and priority
- Returns formatted complaint data

`apply_correction(input_text: str, current_data: Dict)`

- Takes existing complaint data and new input
- Uses AI to validate and correct data
- Ensures data consistency
- Returns corrected complaint data

Features

- **Document Processing:** Extracts text from PDF and DOCX files
- **LLM Integration:** Uses Groq API via LangChain
- **Error Handling:** Graceful fallback for API failures
- **Async Processing:** Non-blocking request handling

Configuration

Set your Groq API key in `.env`:

```
GROQ_API_KEY=your_key_here
```

Frontend Components

App.tsx

Main application component with split-view layout:

- Left: Complaint form
- Right: AI Copilot panel

ComplaintForm.tsx

Form for creating and editing complaints with:

- Input fields for complaint details
- Dropdown selections for categories
- File upload capabilities
- Form validation
- Redux state integration

AICopilot.tsx

AI assistant panel featuring:

- Chat interface for complaint processing
- File upload (PDF/DOCX)
- Real-time AI suggestions
- Integration with backend AI endpoints
- Message history display

ErrorBoundary.tsx

Error handling wrapper that:

- Catches React component errors
- Prevents app crashes
- Displays user-friendly error messages
- Logs errors for debugging

Configuration

Backend Configuration

CORS Settings (`main.py`)

```
ALLOWED_ORIGINS = [  
    "http://localhost:3000",  
    "http://127.0.0.1:3000",  
]
```

Modify to allow additional frontend URLs in production.

Database (`models.py`)

- Uses SQLAlchemy for ORM
- Supports MySQL with PyMySQL
- Auto-creates database if missing
- Can be switched to PostgreSQL by changing `DATABASE_URL`

Frontend Configuration

API Base URL (`src/services/api.ts`)

- Default: `http://localhost:8000`
- Update for different backend URL

Redux Store (`src/store/store.ts`)

- State slices: `complaintSlice`, `uiSlice`
- Custom hooks: `useAppDispatch`, `useAppSelector`

Troubleshooting

Backend Issues

Database Connection Error

```
Error: Can't connect to MySQL server
```

Solution: Ensure MySQL is running and credentials in `.env` are correct

Module Import Error

```
ModuleNotFoundError: No module named 'groq'
```

Solution: Run `pip install -r requirements.txt` in virtual environment

Frontend Issues

API Connection Error

```
Failed to connect to backend at http://localhost:8000
```

Solution: Ensure backend is running on port 8000, check CORS settings

Blank Page Solution: Check browser console for errors, ensure all dependencies installed with `npm install`



Development Workflow

1. Create feature branch
2. Make changes to backend or frontend
3. Test locally using development servers
4. Build frontend: `npm run build`
5. Commit changes with descriptive messages
6. Push and create pull request



Additional Resources

- [FastAPI Documentation](#)
- [React Documentation](#)
- [SQLAlchemy ORM](#)
- [LangChain Documentation](#)
- [Groq API](#)



License

This project is provided as-is for demonstration and development purposes.



Support

For issues or questions about the application, please refer to the code comments and API documentation available at `/docs` when the backend is running.

Last Updated: August 2026